

# **LC-EKF VIO on KITTI raw**

백지에서 시작하는 풀 파이프라인 가이드

개념 · 수학 유도 · 코드 분석 · 운용 절차

C++ ROS-free Stereo VIO 학습 노트

2026

## 요 약

이 문서는 06\_cpp\_LC-EKF\_VIO\_kitti\_raw 프로젝트를 **외부 도움 없이** 처음부터 끝까지 굴러보고자 하는 사용자를 위한 통합 가이드다. 다음 네 가지 층위를 한 권에 묶었다:

1. **개념 (Why):** VIO/SLAM/EKF 가 무엇이고 왜 필요한가, 왜 스테레오 + IMU 조합인가, 왜 LC-EKF 구조인가.
2. **수학 (What):** SO(3) 회전 표현, error-state EKF, RK4 적분, F/G/Q 행렬의 유도, Kalman gain 의 Joseph form, Stereo triangulation, PnP, Umeyama 정합. 각 수식은 왜 그렇게 쓰는가 까지 짚는다.
3. **코드 (How):** 3개 코어 모듈(StereoTracker, ImuPropagator, LcEKF)과 2개 운용 모듈(KittiRawReader, run\_vio.cpp)에 대한 줄 단위 분석과 수식-코드 1:1 매핑.
4. **운용 (Do):** KITTI 데이터셋 다운로드, 캘리브레이션 추출, YAML 설정, 빌드, 실행, 시각화, 다중 시퀀스 일괄 테스트, 트러블슈팅.

본 가이드를 처음부터 끝까지 따라가면 약 1시간 30분 내에 첫 trajectory plot 을 얻을 수 있고, 그 후 후반부의 튜닝 시나리오로 LC-EKF 의 한계와 가능성을 직접 실험해 볼 수 있다.

# 차 례

## 제 1 장

# 30분 개관 — 백지에서 출발하는 이해

이 장은 VIO·SLAM·EKF·KITTI 를 처음 들어보는 사람이 백지 상태에서 “이게 뭐 하는 거고 내가 뭘 할지” 를 30분 안에 잡도록 쓴 도입부다.

### 1.1 왜 이런 시스템이 필요한가 — 위치를 안다는 문제

자율주행/드론/로봇청소기 모두 “내가 지금 어디 있는가”를 매 0.1 초마다 알아야 한다. 방법별 정밀도/한계:

| 방법           | 정밀도       | 한계                               |
|--------------|-----------|----------------------------------|
| GPS (스마트폰)   | 1 ~ 10 m  | 터널/실내/숲 무력. 1 Hz 라 차량 동역학에 너무 느림 |
| GPS-RTK (정밀) | 1 ~ 10 cm | 기지국 인프라 필요. GPS 신호 끊기면 똑같이 무력    |
| 휠 오도메트리      | 단기 정밀     | 미끄러짐/타이어 압력으로 누적 오차              |
| VIO (이 프로젝트) | 누적 0.5~5% | GPS 없이도 동작, 인프라 불필요, 실시간         |

VIO = Visual-Inertial Odometry — 카메라(visual)와 IMU(inertial) 만으로 6-DoF pose(3D 위치 + 3D 회전)를 매 순간 추정한다. GPS 의존을 줄이고 실내에서도 동작하며, 카메라 + 센서만 있으면 어디서든 자세 추적이 가능하다.

### 1.2 왜 카메라랑 IMU를 같이 쓰는가

각 센서 단독의 약점이 명확하고 서로 상보이기 때문이다.

|    | 카메라(스테레오) 단독                                         | IMU 단독                                           |
|----|------------------------------------------------------|--------------------------------------------------|
| 장점 | 정밀한 절대 위치 (특징점 충분 시 0.1 m). 미터 스케일 자동.               | 100~200 Hz 고주파. 빛/날씨/텍스처 무관.                     |
| 단점 | 빠른 회전 시 모션 블러로 추적 실패. 어두운 터널, 단조 텍스처에서 약함. 10 Hz 한계. | 누적 오차로 발산 (가속도 두 번 적분 → 3~5초에 1 m, 1분에 수십~수백 m). |

**둘을 합치면:** 카메라가 절대 기준 역할을 해 IMU 드리프트를 매 프레임 잡아주고, IMU 가 카메라 사이의 빈 시간을 부드럽게 채워 준다. 한쪽이 일시 실패해도 다른 쪽이 버틴다.

### 1.3 왜 스테레오인가 — 미터 스케일의 출처

모노 카메라 한 개의 본질적 한계: 50 cm 떨어진 작은 사과와 5 m 떨어진 큰 사과를 픽셀로는 구별할 수 없다. 이를 **스케일 모호성(scale ambiguity)** 이라 한다.

스테레오 두 카메라:

- 좌↔우 픽셀 차이(*disparity*) + 미리 측정한 두 카메라 사이 거리(**baseline**)
- 깊이 공식: 
$$Z = \frac{f_x \cdot b}{d}$$
- 우리 KITTI calib 의 baseline = **53.7 cm** (KITTI 측이 레이저로 잰 물리값)

이 한 값에서 모든 미터 스케일이 파생된다. 좌측이 cam0 (image\_00), 우측이 cam1 (image\_01). 컬러 페어(image\_02/03)도 있지만 본 코드는 **흑백 스테레오 페어** 만 사용한다 — KLT 추적이 컬러 보간 잡음에 덜 흔들리기 때문이다.

### 1.4 왜 EKF인가 — 두 측정값을 통계적으로 섞기

**EKF (Extended Kalman Filter)** 는 비선형 시스템에서 각 센서의 잡음 모델을 들고 가중평균 하는 표준 도구다.

IMU 적분으로 위치는 (10.0, 5.0, 0.1) 일 것이다 (불확실성  $\pm 0.5$  m)  
VO가 측정한 위치는 (10.5, 4.7, 0.2) 다 (잡음  $\pm 0.1$  m)  
↓ EKF 가  $\sigma$  로 가중해 합침  
통합 추정: (10.45, 4.73, 0.19) ← VO 에 더 가중치 (잡음 작음)

내부적으로 **15차원 상태** (위치 3 + 속도 3 + 자세 3 + 자이로 bias 3 + 가속도 bias 3) 와 그에 대응하는  $15 \times 15$  공분산 행렬 **P** 를 들고 다닌다.

본 프로젝트는 **LC-EKF (Loosely-Coupled)** 구조 — VO 의 출력 위치를 측정값으로 받는다. 더 정밀한 변종은 **Tight MSCKF** (특징점 픽셀을 직접 측정값으로) 로, 본 프로젝트의 다음 마일스톤이다.

### 1.5 KITTI 데이터셋이 뭐고 왜 쓰나

- 독일 KIT (카를스루에 공과대) + 미국 TTIC (토요타 시카고 연구소) 가 2012년 공개
- 차량에 장착: 흑백 스테레오 + 컬러 스테레오 + Velodyne LiDAR + **OXTS RT3003 GPS/IMU 통합 유닛**
- 도시/주거지/도로/고속도로/캠퍼스 다양한 환경 50 시간 +
- **VIO/SLAM 알고리즘 평가의 사실상 표준 벤치마크**

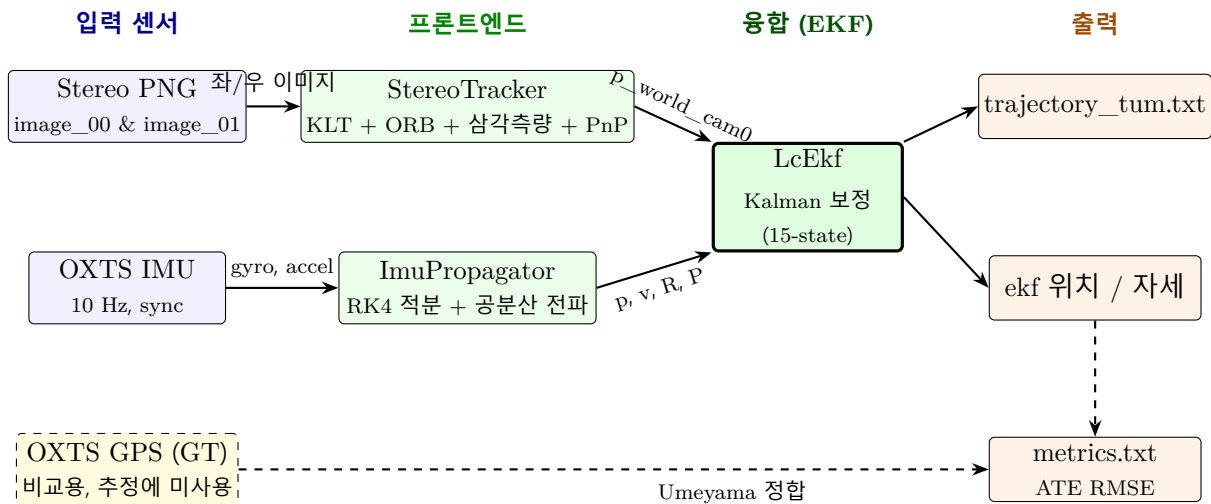
본 프로젝트가 쓰는 부분:

- image\_00, image\_01 — 흑백 스테레오 (원/오)
- oxts/data/\*.txt — 한 줄에 30개 값 (위경도/고도/자세/속도/가속도/각속도/...)

- values[14..16] → 가속도 (body frame, m/s<sup>2</sup>)
- values[20..22] → 각속도 (body frame, rad/s)
- lat/lon/alt → Mercator 투영 → GT 위치

- LiDAR (velodyne\_points/) 와 tracklets 는 사용 안 함

## 1.6 시스템 동작을 한 그림으로



실선 = 데이터 흐름, 점선 = GT 비교만 (추정 단계엔 사용 안 함)

## 1.7 처음부터 끝까지 무엇을 하나 — 5단계 1시간 30분

| 단계 | 내용                                  | 시간    | 가이드         |
|----|-------------------------------------|-------|-------------|
| 1  | WSL Ubuntu + apt 의존성 설치             | 30 분  | 운용 절차       |
| 2  | KITTI 계정 → calib + 1 drive sync 다운  | 30 분  | 운용 절차       |
| 3  | calib 3 종 → $T_{cam,imu}$ 추출 → yaml | 15 분  | 캘리브레이션/YAML |
| 4  | cmake + make → build/run_vio        | 5 분   | 빌드 & 실행     |
| 5  | 단일 시퀀스 실행 + plot 확인                 | 1~5 분 | 실행/시각화      |

여기까지 약 1시간 30분 이면 첫 결과가 나온다. 그 후 확장: 더 긴 시퀀스, 파라미터 튜닝, 다중 시퀀스 비교.

## 1.8 첫 결과 — “정상”의 기준

drive\_0001 (11초, 108프레임) 으로 처음 돌리면 stdout 끝에:

```

1 =====
2 Frames processed : 108
3 ATE RMSE       : 156 cm
4 =====
  
```

ATE RMSE 1~3 m 가 LC-EKF 정상 범위. 같은 시퀀스를 Tight MSCKF (OpenVINS 등) 로 돌리면 5~30 cm 까지 줄어든다 (5~30배 차이). 이게 LC-EKF 의 **본질적 천장**이다.

trajectory\_plot.png 의 왼쪽 패널 (XY top-down) 에서 추정(파란)과 GT(초록)이 **같은 모양**으로 겹쳐 보이면 정상. 약간의 평행이동/회전 오차는 LC-EKF 의 알려진 한계 — 글로벌 yaw 관측이 없어 절대 방향이 점점 어긋난다.

## 1.9 용어집

| 용어            | 의미                                                        |
|---------------|-----------------------------------------------------------|
| VIO           | Visual-Inertial Odometry. 카메라+IMU 위치 추적                   |
| VO            | Visual Odometry. 이미지만으로 카메라 위치 추적                         |
| SLAM          | Simultaneous Localization And Mapping. 위치 + 지도 동시 추정      |
| EKF           | Extended Kalman Filter. 비선형 상태 추정의 표준 도구                  |
| LC-EKF        | Loosely-Coupled EKF. VO 의 출력(위치)을 측정값으로 받음 (이 프로젝트)       |
| Tight MSCKF   | 특징점 픽셀을 직접 측정값으로 받음 (OpenVINS, MSCKF)                     |
| Stereo        | 좌/우 두 카메라로 깊이 추정                                          |
| Baseline      | 두 카메라 사이 물리 거리 (KITTI: 53.7 cm)                           |
| Disparity     | 같은 점이 좌↔우 이미지에서 픽셀 차이                                     |
| KLT           | Kanade-Lucas-Tomasi 광류. 프레임간 특징점 추적 (작은 이동)               |
| ORB           | Oriented FAST + Rotated BRIEF. 회전·스케일 강건 디스크립터 (스테레오 매칭용) |
| FAST          | Features from Accelerated Segment Test. 빠른 코너 검출기         |
| PnP           | Perspective-n-Point. 2D-3D 대응에서 카메라 pose 계산               |
| RANSAC        | RANdom SAMple Consensus. outlier 강건 추정                    |
| IMU           | Inertial Measurement Unit. 3축 가속도 + 3축 자이로                |
| OXTS          | KITTI에 장착된 GPS+IMU 통합 유닛 (RT3003 모델)                      |
| GT            | Ground Truth. 정답값 (KITTI 는 OXTS GPS+RTK)                  |
| ATE           | Absolute Trajectory Error. 정합 후 잔차 RMSE                   |
| RMSE          | Root Mean Square Error                                    |
| TUM 포맷        | timestamp + xyz + 쿼터니언 한 줄씩의 trajectory 저장 형식             |
| Rectification | 스테레오 좌/우 이미지의 epipolar line 을 수평 스캔라인으로 만드는 변환            |
| Lever arm     | IMU 와 카메라 위치 사이의 물리 벡터 ( $p_{IC}$ )                       |
| 6-DoF pose    | 3D 위치 + 3D 회전 = 6 자유도 자세                                  |
| RK4           | Runge-Kutta 4 차. 미분방정식 수치 적분기                             |
| Umeyama       | 두 점군을 강체 정합하는 SVD 기반 알고리즘                                 |

## 제 2 장

# 수학적 배경 — 왜 그 수식인가

이 장은 각 수식이 왜 그렇게 쓰여 있는가를 설명한다. 도출 과정 자체보다는 설계 선택의 이유에 초점이 있다.

### 2.1 회전 표현: $SO(3)$ vs 오일러각 vs 쿼터니언

#### 2.1.1 왜 오일러각이 아닌가

오일러각  $(\phi, \theta, \psi)$  은 직관적이지만 두 가지 본질적 결함:

1. **Gimbal lock**: pitch  $\theta = \pm\pi/2$  에서 yaw 와 roll 이 같은 자유도가 되어 한 자유도가 사라진다. 차량은  $\pm\pi/2$  까지 갈 일이 거의 없지만, 드론/로봇팔에선 빈번하다.
2. **회전 합성이 비선형**:  $R(\phi_1, \theta_1, \psi_1) \cdot R(\phi_2, \theta_2, \psi_2)$  의 결과를 다시 오일러각으로 뽑으려면 비자명한 식이 필요하다. EKF 에서  $R \leftarrow R \cdot R_\delta$  같은 합성을 매 스텝 해야 하는데, 이 비선형성이 누적된다.

#### 2.1.2 왜 쿼터니언만으로는 부족한가

쿼터니언  $q = (q_w, q_x, q_y, q_z)$  은 4개 값으로 회전을 나타내며 gimbal lock 이 없다. 그러나:

- **유닛 제약**  $\|q\| = 1$ : 4개 값이지만 자유도는 3. EKF 가 4D 상태로 쿼터니언을 추정하면 매 업데이트마다 정규화로 매니폴드 위로 끌어와야 하고, 공분산이 4D 인데 실제 자유도가 3D 라 수치적 비조건수(numerical ill-conditioning) 가 생긴다.
- **이중 표지(double cover)**:  $q$  와  $-q$  는 같은 회전이라 부호 모호성이 있다.

#### 2.1.3 왜 $SO(3)$ 와 error-state 가 정답인가

$SO(3)$  는  $3 \times 3$  직교행렬  $R$  로서  $R^T R = I, \det R = 1$  인 3차원 매니폴드이다. 그 **접공간(tangent space)** 은 3차원 Lie algebra  $\mathfrak{so}(3)$  으로, 작은 각도 벡터  $\delta\theta \in \mathbb{R}^3$  으로 표현된다.

이걸 활용한 **error-state EKF** 의 핵심 아이디어:

- **Nominal state**: 큰 회전을  $R \in SO(3)$  ( $3 \times 3$  행렬) 로 그대로 들고 다닌다.
- **Error state**: 작은 회전 오차만  $\delta\theta \in \mathbb{R}^3$  로 추정한다 — 이 부분이 선형 EKF 가 다룬다.
- **합성**: 보정 시  $R \leftarrow R \cdot \text{Exp}(\delta\theta)$  로 매니폴드 위에 정확히 머물게 한다.

이 분리로 얻는 것:

1. EKF 가 다루는 차원이 15 (4D 쿼터니언 시 16) 로 정확히 자유도와 일치 → 공분산이 항상 well-conditioned.
2.  $R$  자체는 매니폴드를 절대 벗어나지 않음 (정규화 후 cosmetics 만 필요).
3. Jacobian 이 단순해짐 (3D 작은 각도에 대한 미분).

#### 2.1.4 핵심 SO(3) 연산

**Hat 연산자**  $[\cdot]_{\times} : \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$ :

$$[\mathbf{v}]_{\times} = \begin{bmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & -v_1 \\ -v_2 & v_1 & 0 \end{bmatrix} \quad (2.1)$$

성질:  $[\mathbf{a}]_{\times} \mathbf{b} = \mathbf{a} \times \mathbf{b}$ . 외적을 행렬 곱으로 바꾸는 도구.

**지수 사상 (Rodrigues 공식)**  $\text{Exp} : \mathbb{R}^3 \rightarrow \text{SO}(3)$ :

$$\text{Exp}(\boldsymbol{\omega}) = I + \frac{\sin \theta}{\theta} [\boldsymbol{\omega}]_{\times} + \frac{1 - \cos \theta}{\theta^2} [\boldsymbol{\omega}]_{\times}^2, \quad \theta = \|\boldsymbol{\omega}\| \quad (2.2)$$

작은  $\boldsymbol{\omega}$  에서  $\text{Exp}(\boldsymbol{\omega}) \approx I + [\boldsymbol{\omega}]_{\times}$ . 즉 *first-order Taylor* 가 정확히 작은 회전 이다.

**로그 사상**  $\text{Log} : \text{SO}(3) \rightarrow \mathbb{R}^3$ :  $R$  에서 회전축×각도 벡터를 추출.

이들이 `include/core/so3.hpp` 에 구현돼 있고, 모든 회전 합성과 미분에서 일관되게 쓰인다.

## 2.2 Error-state EKF 의 15차원 상태

### 2.2.1 Nominal state → Error state 분해

**Nominal state** (실제 추정치):

$$\mathbf{x} = (\mathbf{p}, \mathbf{v}, R, \mathbf{b}_g, \mathbf{b}_a) \in \mathbb{R}^3 \times \mathbb{R}^3 \times \text{SO}(3) \times \mathbb{R}^3 \times \mathbb{R}^3 \quad (2.3)$$

**Error state** (작은 편차):

$$\delta \mathbf{x} = (\delta \mathbf{p}, \delta \mathbf{v}, \delta \boldsymbol{\theta}, \delta \mathbf{b}_g, \delta \mathbf{b}_a) \in \mathbb{R}^{15} \quad (2.4)$$

합성 규칙:

$$\begin{aligned} \mathbf{p}_{\text{true}} &= \mathbf{p} + \delta \mathbf{p} \\ \mathbf{v}_{\text{true}} &= \mathbf{v} + \delta \mathbf{v} \\ R_{\text{true}} &= R \cdot \text{Exp}(\delta \boldsymbol{\theta}) \\ \mathbf{b}_{g,\text{true}} &= \mathbf{b}_g + \delta \mathbf{b}_g \\ \mathbf{b}_{a,\text{true}} &= \mathbf{b}_a + \delta \mathbf{b}_a \end{aligned}$$

위치/속도/바이어스는 가법, 회전만 Exp 합성.

## 2.2.2 왜 IMU 측정 모델이 이렇게 생겼나

IMU 가 출력하는 raw 신호:

$$\tilde{\omega} = \omega + b_g + n_g \quad (2.5)$$

$$\tilde{a} = R^\top(a - g) + b_a + n_a \quad (2.6)$$

핵심 포인트:

- 가속도계는 비중력 가속도만 잰다 (자유낙하 시 0 출력). 그래서 "world frame 가속도  $a$ " 에서 "world frame 중력  $g$ " 을 빼고 body frame 으로 회전 변환한 게 raw 출력.
- 따라서 정지 상태 의 IMU 가속도계는  $R^\top(-g)$  를 출력 — 즉 "중력의 반대 방향" 을 가리킨다. 이게 초기 자세 추정의 기반이다.

본 코드의 초기화 (run\_vio.cpp 의 initialize\_from\_imu) 가 정확히 이 원리를 쓴다: 1초 동안 IMU 평균을 내고, 그 평균 가속도가  $R^\top \cdot (0, 0, 9.81)^\top$  이 되도록  $R_0$  를 계산한다.

## 2.2.3 왜 15차원인가 (16 도 17 도 아니고)

자유도 회계:

- 위치 3 + 속도 3 = 6
- 회전 3 (SO(3) 의 차원)
- 자이로 bias 3 + 가속도 bias 3 = 6
- **합계 15**

쿼터니언으로 표현했다면 16D 가 되지만 자유도는 여전히 15. SO(3) 표현이 자유도와 정확히 일치한다.

## 2.3 IMU 운동학과 RK4 적분

### 2.3.1 연속시간 운동학

bias 보정된 IMU 측정값을  $\omega_c = \tilde{\omega} - b_g$ ,  $a_c = \tilde{a} - b_a$  로 두면:

$$\dot{p} = v \quad (2.7)$$

$$\dot{v} = R a_c + g \quad (2.8)$$

$$\dot{R} = R[\omega_c]_\times \quad (2.9)$$

(bias 는 random walk 가정:  $\dot{b}_g = n_{bg}$ ,  $\dot{b}_a = n_{ba}$ )

### 2.3.2 왜 RK4 인가

가장 단순한 forward Euler:  $p_{k+1} = p_k + dt \cdot v_k$  는 1차 정확도라 오차가  $O(dt)$  로 누적된다. IMU 가 10 Hz ( $dt = 0.1$  s) 만 되어도 1초 동안 가속이 변하면 무시 못 할 오차가 쌓인다.

RK4 (Runge-Kutta 4차) 는 한 스텝 안에서 4번 도함수를 평가해  $O(dt^5)$  정확도를 낸다 (지역 오차). 같은  $dt = 0.1$  s 에서 forward Euler 대비  $\sim 10^4$  배 정확.

본 코드 (imu\_propagator.cpp:40-66) 의 rk4\_step:

```

1 void ImuPropagator::rk4_step(double dt,
2                               const Vector3d& gyro_c,
3                               const Vector3d& accel_c) {
4     State s0{p, v, R};
5     auto k1 = state_derivative(s0, gyro_c, accel_c);
6
7     State s1{p + 0.5*dt*k1.p,
8              v + 0.5*dt*k1.v,
9              R * so3::Exp(0.5*dt*gyro_c)};
10    auto k2 = state_derivative(s1, gyro_c, accel_c);
11
12    State s2{p + 0.5*dt*k2.p,
13             v + 0.5*dt*k2.v,
14             R * so3::Exp(0.5*dt*gyro_c)};
15    auto k3 = state_derivative(s2, gyro_c, accel_c);
16
17    State s3{p + dt*k3.p,
18             v + dt*k3.v,
19             R * so3::Exp(dt*gyro_c)};
20    auto k4 = state_derivative(s3, gyro_c, accel_c);
21
22    p = p + (dt/6.0) * (k1.p + 2*k2.p + 2*k3.p + k4.p);
23    v = v + (dt/6.0) * (k1.v + 2*k2.v + 2*k3.v + k4.v);
24    R = R * so3::Exp(dt * gyro_c);
25    R = Eigen::Quaterniond(R).normalized().toRotationMatrix();
26 }

```

Listing 2.1: RK4 한 스텝 — src/ekf/imu\_propagator.cpp

**한 가지 단순화:** 회전은  $R \cdot \text{Exp}(dt \omega_c)$  로 한 번에 적용한다 (RK4 수준의 회전은 Magnus expansion 이 필요한데, 차량 동역학에선 무시 가능).

## 2.4 공분산 전파: F, G, Q 행렬 유도

### 2.4.1 Error-state ODE

$\delta x$  의 시간 도함수를 nominal state 주변에서 1차 선형화하면:

$$\dot{\delta x} = F \delta x + G n \quad (2.10)$$

여기서  $n = (n_g, n_a, n_{bg}, n_{ba}) \in \mathbb{R}^{12}$  는 잡음 벡터.

### 2.4.2 F 행렬 — 각 블록의 유도

**상태 정의:**  $\delta x = [\delta p, \delta v, \delta \theta, \delta b_g, \delta b_a] \in \mathbb{R}^{15}$ . Right-perturbation 규약  $R_{\text{true}} = \hat{R} \text{Exp}(\delta \theta)$  를 채택한다.

**블록별 유도**

(1)  $\dot{\delta p} = \delta v \quad \Rightarrow F_{pp} = 0, F_{pv} = I$

위치의 시간 미분은 속도이므로 자명하다.

(2)  $\dot{\delta v}$  유도  $\Rightarrow F_{v\theta} = -R[a_c]_{\times}, F_{vb_a} = -R$

진실 속도 미분과 nominal 속도 미분의 차:

$$\begin{aligned}\dot{\mathbf{v}}_{\text{true}} &= R_{\text{true}} \mathbf{a}_{c,\text{true}} + \mathbf{g} = \hat{R} \text{Exp}(\delta\boldsymbol{\theta}) (\mathbf{a}_c - \delta\mathbf{b}_a + \mathbf{n}_a) + \mathbf{g} \\ \dot{\mathbf{v}} &= \hat{R} \mathbf{a}_c + \mathbf{g}\end{aligned}$$

$\text{Exp}(\delta\boldsymbol{\theta}) \approx I + [\delta\boldsymbol{\theta}]_{\times}$  로 1차 전개하면:

$$\delta\dot{\mathbf{v}} \approx \hat{R} [\delta\boldsymbol{\theta}]_{\times} \mathbf{a}_c - \hat{R} \delta\mathbf{b}_a + \hat{R} \mathbf{n}_a \quad (2.11)$$

$[\delta\boldsymbol{\theta}]_{\times} \mathbf{a}_c = \delta\boldsymbol{\theta} \times \mathbf{a}_c = -[\mathbf{a}_c]_{\times} \delta\boldsymbol{\theta}$  이므로:

$$\delta\dot{\mathbf{v}} = \underbrace{-R[\mathbf{a}_c]_{\times}}_{F_{v\theta}} \delta\boldsymbol{\theta} + \underbrace{(-R)}_{F_{vb_a}} \delta\mathbf{b}_a + R \mathbf{n}_a \quad (2.12)$$

(3)  $\delta\dot{\boldsymbol{\theta}}$  유도  $\Rightarrow F_{\theta\theta} = -[\boldsymbol{\omega}_c]_{\times}, F_{\theta b_g} = -R$   
회전 운동 방정식  $\dot{R} = R[\boldsymbol{\omega}]_{\times}$  에서 right-perturbation으로 전개하면:

$$\frac{d}{dt}(\hat{R} \text{Exp}(\delta\boldsymbol{\theta})) = \hat{R} \text{Exp}(\delta\boldsymbol{\theta}) [\boldsymbol{\omega}_c - \delta\mathbf{b}_g + \mathbf{n}_g]_{\times} \quad (2.13)$$

좌변의 product rule 과  $\text{Exp}(\delta\boldsymbol{\theta}) \approx I + [\delta\boldsymbol{\theta}]_{\times}$  를 적용하면:

$$\hat{R} \text{Exp}(\delta\boldsymbol{\theta}) + \hat{R} \text{Exp}(\delta\boldsymbol{\theta}) [\delta\dot{\boldsymbol{\theta}}]_{\times} \approx \hat{R} (I + [\delta\boldsymbol{\theta}]_{\times}) [\boldsymbol{\omega}_c - \delta\mathbf{b}_g + \mathbf{n}_g]_{\times} \quad (2.14)$$

$\hat{R} = \hat{R}[\boldsymbol{\omega}_c]_{\times}$  를 대입하고 양변에  $\hat{R}^{-1}$  를 곱한 뒤 1차 항만 취하면:

$$[\delta\dot{\boldsymbol{\theta}}]_{\times} \approx -[\boldsymbol{\omega}_c]_{\times} [\delta\boldsymbol{\theta}]_{\times} + [\delta\boldsymbol{\theta}]_{\times} [\boldsymbol{\omega}_c]_{\times} - [\delta\mathbf{b}_g]_{\times} + [\mathbf{n}_g]_{\times} \quad (2.15)$$

교환자  $[[a]_{\times}, [b]_{\times}] = [a \times b]_{\times}$  를 정리하면 벡터 형태로:

$$\delta\dot{\boldsymbol{\theta}} = \underbrace{-[\boldsymbol{\omega}_c]_{\times}}_{F_{\theta\theta}} \delta\boldsymbol{\theta} + \underbrace{(-R)}_{F_{\theta b_g}} \delta\mathbf{b}_g + R \mathbf{n}_g \quad (2.16)$$

$F_{\theta b_g} = -R$  인 이유: bias 는 body frame 잡음이고  $\delta\boldsymbol{\theta}$  는 world frame 표현이라  $R$  이 필요하다.

(4) bias 행: random walk 모델  $\dot{\mathbf{b}}_{g/a} = \mathbf{n}_{bg/ba}$  에서 F 의 bias 행은 모두 0.

이를 합치면 15×15 연속 전이 행렬:

$$F = \begin{bmatrix} \mathbf{0} & I & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & -R[\mathbf{a}_c]_{\times} & \mathbf{0} & -R \\ \mathbf{0} & \mathbf{0} & -[\boldsymbol{\omega}_c]_{\times} & -R & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{array}{l} \leftarrow \delta\mathbf{p} \\ \leftarrow \delta\mathbf{v} \\ \leftarrow \delta\boldsymbol{\theta} \\ \leftarrow \delta\mathbf{b}_g \\ \leftarrow \delta\mathbf{b}_a \end{array} \quad (2.17)$$

코드 매핑 (imu\_propagator.cpp:93-99):

| 블록                               | 코드                                                     | 의미                             |
|----------------------------------|--------------------------------------------------------|--------------------------------|
| $F[0, 3] = I$                    | <code>F.block&lt;3,3&gt;(0,3) = I</code>               | $\dot{\delta p} = \delta v$    |
| $F[3, 6] = -R[a_c]_{\times}$     | <code>F.block&lt;3,3&gt;(3,6) = -R*hat(accel_c)</code> | 자세 오차 $\rightarrow$ 속도 미분      |
| $F[3, 12] = -R$                  | <code>F.block&lt;3,3&gt;(3,12) = -R</code>             | accel bias $\rightarrow$ 속도 미분 |
| $F[6, 6] = -[\omega_c]_{\times}$ | <code>F.block&lt;3,3&gt;(6,6) = -hat(gyro_c)</code>    | 자세 error 자체 dynamics           |
| $F[6, 9] = -R$                   | <code>F.block&lt;3,3&gt;(6,9) = -R</code>              | gyro bias $\rightarrow$ 자세 미분  |

### 2.4.3 G 행렬 — 잡음 입력 매핑

잡음 벡터  $\mathbf{n} = (\mathbf{n}_g, \mathbf{n}_a, \mathbf{n}_{bg}, \mathbf{n}_{ba}) \in \mathbb{R}^{12}$  가 error state 에 어떻게 들어오는지 위 유도에서 직접 읽을 수 있다:

- $\delta \dot{\mathbf{v}}$  항의  $R \mathbf{n}_a$ : accel 잡음은 body frame ( $\mathbf{n}_a$  단위:  $\text{m/s}^2/\sqrt{\text{Hz}}$ ), 하지만  $\delta \mathbf{v}$  는 world frame  $\rightarrow R$  로 회전 필요.
- $\delta \dot{\boldsymbol{\theta}}$  항의  $R \mathbf{n}_g$ : gyro 잡음은 body frame ( $\mathbf{n}_g$  단위:  $\text{rad/s}/\sqrt{\text{Hz}}$ ),  $\delta \boldsymbol{\theta}$  는 world frame  $\rightarrow$  마찬가지로  $R$  필요.
- bias random walk  $\mathbf{n}_{bg}, \mathbf{n}_{ba}$  는 각 bias 에 직접 더해지므로  $I$ .

따라서  $G \in \mathbb{R}^{15 \times 12}$ :

$$G = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & R & \mathbf{0} & \mathbf{0} \\ R & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & I & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & I \end{bmatrix} \quad \begin{array}{l} \leftarrow \delta \mathbf{p} \\ \leftarrow \delta \mathbf{v} \\ \leftarrow \delta \boldsymbol{\theta} \\ \leftarrow \delta \mathbf{b}_g \\ \leftarrow \delta \mathbf{b}_a \end{array} \quad \begin{array}{l} \uparrow \mathbf{n}_g \\ \uparrow \mathbf{n}_a \\ \uparrow \mathbf{n}_{bg} \\ \uparrow \mathbf{n}_{ba} \end{array} \quad (2.18)$$

코드 매핑 (`imu_propagator.cpp:106-110`):

| 블록 (행, 열 인덱스)            | 코드                                        | 의미                                               |
|--------------------------|-------------------------------------------|--------------------------------------------------|
| $G[3 : 6, 3 : 6] = R$    | <code>G.block&lt;3,3&gt;(3,3) = R</code>  | accel 잡음 $\rightarrow \delta \mathbf{v}$         |
| $G[6 : 9, 0 : 3] = R$    | <code>G.block&lt;3,3&gt;(6,0) = R</code>  | gyro 잡음 $\rightarrow \delta \boldsymbol{\theta}$ |
| $G[9 : 12, 6 : 9] = I$   | <code>G.block&lt;3,3&gt;(9,6) = I</code>  | gyro walk $\rightarrow \delta \mathbf{b}_g$      |
| $G[12 : 15, 9 : 12] = I$ | <code>G.block&lt;3,3&gt;(12,9) = I</code> | accel walk $\rightarrow \delta \mathbf{b}_a$     |

### 2.4.4 $Q_c$ 행렬 — 연속시간 잡음 공분산

잡음 모델은 white noise 밀도:  $\mathbf{n}_g \sim \mathcal{N}(\mathbf{0}, \sigma_g^2 I)$  등 (단위  $\sigma/\sqrt{\text{Hz}}$ , 즉 PSD 형태).  $12 \times 12$  대각 행렬:

$$Q_c = \text{diag}(\underbrace{\sigma_g^2 I_3}_{\mathbf{n}_g}, \underbrace{\sigma_a^2 I_3}_{\mathbf{n}_a}, \underbrace{\sigma_{bg}^2 I_3}_{\mathbf{n}_{bg}}, \underbrace{\sigma_{ba}^2 I_3}_{\mathbf{n}_{ba}}) \in \mathbb{R}^{12 \times 12} \quad (2.19)$$

각  $\sigma$  는 YAML 의 `imu_noise` 섹션에서 읽히며 (`imu_propagator.cpp:112-119`):

| 파라미터          | YAML 키      | 단위                                      | $Q_c$ 대각 블록         |
|---------------|-------------|-----------------------------------------|---------------------|
| $\sigma_g$    | gyro_noise  | rad/s/ $\sqrt{\text{Hz}}$               | $\sigma_g^2 I_3$    |
| $\sigma_a$    | accel_noise | m/s <sup>2</sup> / $\sqrt{\text{Hz}}$   | $\sigma_a^2 I_3$    |
| $\sigma_{bg}$ | gyro_walk   | rad/s <sup>2</sup> / $\sqrt{\text{Hz}}$ | $\sigma_{bg}^2 I_3$ |
| $\sigma_{ba}$ | accel_walk  | m/s <sup>3</sup> / $\sqrt{\text{Hz}}$   | $\sigma_{ba}^2 I_3$ |

#### 2.4.5 이산화: $\Phi$ 와 $Q_d$

연속 ODE 를  $dt$  동안 적분 (imu\_propagator.cpp:101-124):

$$\Phi \approx I + F dt \quad (1\text{차 Euler 근사}) \quad (2.20)$$

$$Q_d = G Q_c G^\top \cdot dt \quad (\text{잡음 공분산 이산화}) \quad (2.21)$$

**왜  $dt$  를 곱하나:**  $Q_c$  는 스펙트럼 밀도 (단위  $\sigma^2/\text{Hz}$ ) 이므로, 이를  $dt$  구간에서 적분하면  $\sigma^2 \cdot dt$  가 분산이 된다. 구체적으로  $Q_d$  의  $(i, j)$  블록은:

$$[Q_d]_{ij} = \sigma_i^2 \cdot [GG^\top]_{ij} \cdot dt \quad (2.22)$$

전체 공분산 전파 (imu\_propagator.cpp:124):

$$P_{k+1} = \Phi P_k \Phi^\top + Q_d \quad (2.23)$$

코드에서는 `Phi = I + F*dt, Qd = G*Qc*G.transpose()*dt` 로 계산한 뒤 한 줄로 적용:

```
1 P = Phi * P * Phi.transpose() + Qd;
```

Listing 2.2: imu\_propagator.cpp:124

## 2.5 측정 모델과 Kalman 보정

### 2.5.1 LC-EKF 측정 모델

VO 가 출력한 cam0 의 world frame 위치  $p_{\text{cam0}}^W$  를 측정값으로:

$$z = p_{\text{cam0}}^W = R_I^W p_{IC} + p_I^W + n_v \quad (2.24)$$

여기서:

- $p_I^W$  = IMU 의 world frame 위치 (EKF state).
- $R_I^W$  = world  $\leftarrow$  body 회전 (EKF state).
- $p_{IC}$  = body frame 에서 본 cam0 원점 (**lever arm**, calibration 으로 고정).
- $n_v$  = VO 측정 잡음,  $\sim \mathcal{N}(\mathbf{0}, \sigma_{v0}^2 I_3)$ .

### 2.5.2 H 행렬 유도 — lc\_ekf.cpp:38-43

예측 측정값  $h(\mathbf{x}) = R_I^W \mathbf{p}_{IC} + \mathbf{p}_I^W$  를 error state 로 선형화한다. Right-perturbation:

$$R_{\text{true}} = \hat{R}_I^W \text{Exp}(\delta\boldsymbol{\theta}), \quad \mathbf{p}_{\text{true}} = \hat{\mathbf{p}}_I^W + \delta\mathbf{p} \quad (2.25)$$

예측값에서의 편차:

$$\begin{aligned} \delta h &= h(\hat{\mathbf{x}} \oplus \delta\mathbf{x}) - h(\hat{\mathbf{x}}) \\ &= \hat{R} \text{Exp}(\delta\boldsymbol{\theta}) \mathbf{p}_{IC} + \hat{\mathbf{p}} + \delta\mathbf{p} - (\hat{R} \mathbf{p}_{IC} + \hat{\mathbf{p}}) \\ &\approx \hat{R} (I + [\delta\boldsymbol{\theta}]_{\times}) \mathbf{p}_{IC} - \hat{R} \mathbf{p}_{IC} + \delta\mathbf{p} \\ &= \hat{R} [\delta\boldsymbol{\theta}]_{\times} \mathbf{p}_{IC} + \delta\mathbf{p} \end{aligned} \quad (2.26)$$

핵심 단계:  $[\delta\boldsymbol{\theta}]_{\times} \mathbf{p}_{IC} = \delta\boldsymbol{\theta} \times \mathbf{p}_{IC} = -\mathbf{p}_{IC} \times \delta\boldsymbol{\theta} = -[\mathbf{p}_{IC}]_{\times} \delta\boldsymbol{\theta}$   
따라서:

$$\delta h = \delta\mathbf{p} - \hat{R} [\mathbf{p}_{IC}]_{\times} \delta\boldsymbol{\theta} \quad (2.27)$$

각 편미분:

$$\frac{\partial h}{\partial \delta\mathbf{p}} = I_3 \quad (2.28)$$

$$\frac{\partial h}{\partial \delta\boldsymbol{\theta}} = -R_I^W [\mathbf{p}_{IC}]_{\times} \quad (2.29)$$

$$\frac{\partial h}{\partial \delta\mathbf{v}} = 0, \quad \frac{\partial h}{\partial \delta\mathbf{b}_g} = 0, \quad \frac{\partial h}{\partial \delta\mathbf{b}_a} = 0 \quad (2.30)$$

**주의:**  $-R[\mathbf{p}_{IC}]_{\times} \neq [R\mathbf{p}_{IC}]_{\times}$  (일반적으로 같지 않다). 표준 항등식  $[R\mathbf{v}]_{\times} = R[\mathbf{v}]_{\times} R^{\top}$  에 의해  $-R[\mathbf{p}_{IC}]_{\times} = -[R\mathbf{p}_{IC}]_{\times} R \neq [R\mathbf{p}_{IC}]_{\times}$ . 올바른 야코비안은  $-R[\mathbf{p}_{IC}]_{\times}$  이다.

따라서:

$$H = \begin{bmatrix} I_3 & \mathbf{0} & -R_I^W [\mathbf{p}_{IC}]_{\times} & \mathbf{0} & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{3 \times 15} \quad (2.31)$$

코드 (lc\_ekf.cpp:41-43):

```
1 H.block<3,3>(0,0) = Matrix3d::Identity(); // dh/d(delta_p) = I
2 H.block<3,3>(0,6) = -R_wI * so3::hat(p_IC_); // dh/d(delta_theta) = -R*[p_IC]x
```

**lever arm 영향:** KITTI 에서  $\mathbf{p}_{IC} \approx (-0.3, 0.7, -1.1)^{\top}$  m (약 1.3 m). 이 값이 0 이면  $H_{\theta} = 0$  이 되고, 클수록 자세 오차가 위치 측정에 더 크게 투영된다. 야코비안을 정확히 계산하는 것이 EKF 수렴에 중요하다.

### 2.5.3 Kalman gain 과 상태 보정

표준 EKF 업데이트 (lc\_ekf.cpp:46-56):

$$S = HPH^{\top} + R_{\text{noise}} \quad (\text{innovation covariance}) \quad (2.32)$$

$$K = PH^{\top} S^{-1} \quad (\text{Kalman gain}) \quad (2.33)$$

$$\delta\mathbf{x} = K(\mathbf{z} - h(\mathbf{x})) \quad (\text{error correction}) \quad (2.34)$$

해석:

- $R_{\text{noise}} = \sigma_{\text{vo}}^2 I_3$  가 작을수록 (VO 신뢰)  $K$  가 커진다  $\rightarrow$  보정량 큼.
- $P$  가 클수록 (IMU 의 추정 불확실)  $K$  가 커진다  $\rightarrow$  측정값에 더 의존.
- 둘의 비율이  $K$  의 핵심.

#### 2.5.4 왜 Joseph form 인가

전통적 공분산 업데이트  $P^+ = (I - KH)P$  는 수학적으로 맞지만 수치적으로 위험:

- 부동소수 오차로  $P^+$  가 비대칭이 되거나 음의 eigenvalue 가 생길 수 있음.
- 특히 측정값이 매우 신뢰될 때 ( $R_{\text{noise}} \rightarrow 0$ ) 더 심해짐.

Joseph form (lc\_ekf.cpp:60-61):

$$P^+ = (I - KH)P(I - KH)^T + KR_{\text{noise}}K^T \quad (2.35)$$

이 형태는:

- **대칭성 보장:** 두 항 모두  $ABA^T$  꼴이라 자동으로 대칭.
- **양의 정부호 보장:**  $P, R_{\text{noise}}$  가 PSD 면 결과도 PSD.
- 추가 비용은  $O(n^3)$  한 번 더 (15D 에선 무시 가능).

## 2.6 Stereo Triangulation 과 PnP

### 2.6.1 왜 베이스라인이 미터 스케일을 결정하나

Pinhole + rectified stereo 모델에서 한 점이 좌측 픽셀  $(u_L, v)$ , 우측  $(u_R, v)$  에 보일 때:

$$Z = \frac{f_x \cdot b}{u_L - u_R}, \quad X = \frac{(u_L - c_x)Z}{f_x}, \quad Y = \frac{(v - c_y)Z}{f_y} \quad (2.36)$$

$b$  는 미터 단위 베이스라인. 모든 깊이  $Z$  가 미터로 자동 결정된다. KITTI 의  $b = 0.537$  m 가 calibration 시 레이저로 측정되어 yaml 의  $T_{\text{cam,imu}}$  에 박혀 있다.

코드에서는 OpenCV `cv::triangulatePoints` 가  $P_0, P_1$  projection matrix 를 받아 4D 동차좌표를 출력한다 (stereo\_tracker.cpp).

### 2.6.2 PnP 가 풀어내는 것

$N$  개의 2D-3D 대응  $(u_i, v_i, X_i, Y_i, Z_i)$  에서 카메라 pose  $(R, t)$  를 풀어내는 문제. 최소  $N = 4$  (P3P 는 3 + ambiguity, EPnP/UPnP 는 4 이상 권장).

본 코드는 `cv::solvePnP` 사용 — RANSAC outlier 제거 후 EPnP + reprojection 비선형 최적화.

**왜 단순 LSQ 가 아닌 RANSAC?** KLT/ORB 매칭에서 outlier (잘못된 대응) 가 5~30% 발생. 단순 최소제곱은 outlier 한 개에 휘둘리지만 RANSAC 은 "inlier 다수결" 로 강건하다.

## 2.7 ATE 계산: Umeyama 정합

추정 trajectory  $\{e_i\}$  와 GT  $\{g_i\}$  간 **절대 trajectory 오차**:

**왜 단순  $\|e_i - g_i\|$  가 아닌가?**

EKF 가 임의 시점을 원점으로 잡아 (first IMU sample = (0,0,0)) 시작하면 GT 와 전역  $SE(3)$  변환 만큼 어긋난다. 그 변환을 "드리프트" 로 카운트하면 안 된다.

Umeyama (1991) 알고리즘 — SVD 로 두 점군의 최적 강체정합을 구함:

$$\min_{R, t} \sum_i \|(R e_i + t) - g_i\|^2 \quad (2.37)$$

→  $\Sigma = \frac{1}{N} \sum_i (e_i - \bar{e})(g_i - \bar{g})^\top$  의 SVD  $U \Sigma V^\top$  로:

$$R^* = V \text{diag}(1, 1, \det(VU^\top)) U^\top \quad (2.38)$$

$$t^* = \bar{g} - R^* \bar{e} \quad (2.39)$$

본 코드 (run\_vio.cpp:193) 는 Eigen 의 `Eigen::umeyama(..., false)` 호출 — 두 번째 인자 `false` 는 "스케일 고정" 의미. 우리는 스테레오로 미터 스케일이 이미 맞으니 회전+평행이동만 정합.

ATE RMSE:

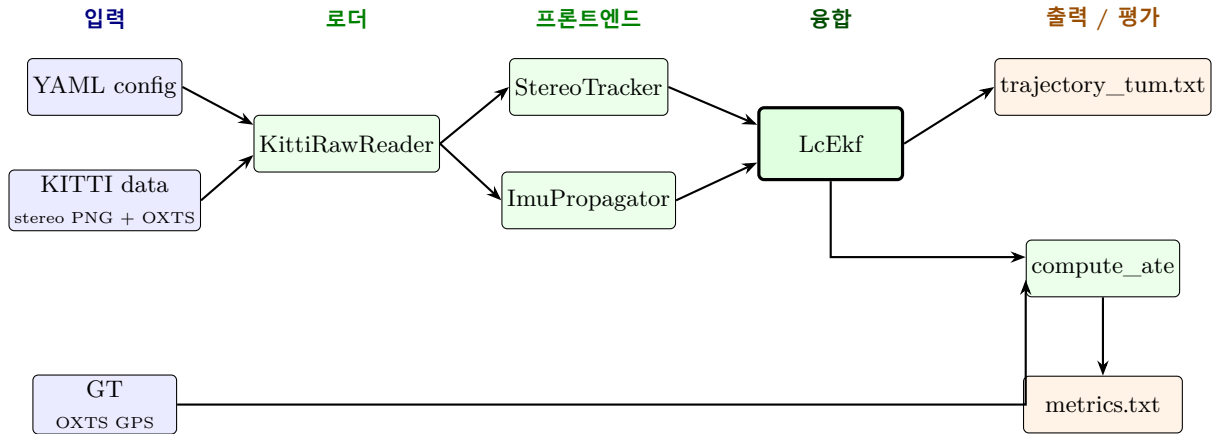
$$\text{ATE}_{\text{RMSE}} = \sqrt{\frac{1}{N} \sum_i \|(R^* e_i + t^*) - g_i\|^2} \quad (2.40)$$

## 제 3 장

# 시스템 아키텍처

### 3.1 전체 흐름

본 프로젝트는 ROS 없이 단일 C++ 실행파일로 구성된다. 입력 → 처리 → 출력의 큰 그림:



### 3.2 발표용 코어 모듈 요약

PPT에서는 전체 코드를 모두 설명하기보다, 아래 3개 코어 모듈이 어떤 정보를 주고받는지 먼저 보여주는 편이 가장 명확하다.

$$\begin{aligned}
 \text{StereoTracker} &\longrightarrow z_k = \mathbf{p}_{C_k}^W & \implies & \text{LcEkf} \\
 \text{ImuPropagator} &\longrightarrow \mathbf{x}_{k+1}^-, \mathbf{P}_{k+1}^- \\
 \mathbf{x} = \{\mathbf{p}, \mathbf{v}, R, \mathbf{b}_g, \mathbf{b}_a\}, & \quad \delta \mathbf{x} = \begin{bmatrix} \delta \mathbf{p} & \delta \mathbf{v} & \delta \boldsymbol{\theta} & \delta \mathbf{b}_g & \delta \mathbf{b}_a \end{bmatrix}^\top \in \mathbb{R}^{15}
 \end{aligned}$$

| 모듈            | 발표용 한 줄                              | PPT 에 넣을 핵심 수식                                                                                                                                                      |
|---------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| StereoTracker | 스테레오 이미지에서 sparse VO 위치 추정 생성        | $\min_{R_{CW}, t_{CW}} \sum_i \ u_i - \pi(K(R_{CW}P_i^W + t_{CW}))\ ^2,$ $z_k = p_{C_k}^W$                                                                          |
| ImuPropagator | IMU 로 nominal state 와 co-variance 예측 | $\dot{p} = v,$ $\dot{v} = R(a_m - b_a) + g,$ $\dot{R} = R[\omega_m - b_g]_{\times}$ $P_{k+1}^- = \Phi P_k^+ \Phi^T + G Q G^T,$ $\Phi \approx I + F \Delta t$        |
| LcEKF         | VO 위치 추정으로 IMU drift 보정              | $h(x) = p_I^W + R_I^W p_{IC},$ $r = z - h(x)$ $H = [I_3 \quad 0 \quad -R_I^W [p_{IC}]_{\times} \quad 0 \quad 0]$ $K = P H^T (H P H^T + R_n)^{-1},$ $\delta x = K r$ |

발표의 핵심 메시지: 이 시스템은 **Tight VIO** 가 아니라 **Loosely-Coupled VIO** 다. 즉 feature residual 을 EKF 에 직접 넣지 않고, StereoTracker 가 만든 camera position 을 EKF measurement 로 사용한다.

### 3.3 모듈 책임 분리

| 모듈             | 위치                              | 책임                                                                              |
|----------------|---------------------------------|---------------------------------------------------------------------------------|
| KittiRawReader | src/io/kitti_raw_reader.cpp     | 디스크에서 KITTI raw 디렉터리 파싱. IMU/CAM/GT 를 std::vector 로 적재.                         |
| StereoTracker  | src/frontend/stereo_tracker.cpp | KITTI ORB + 삼각측량 + PnP 로 cam0 의 6-DoF pose 출력. 매니폴드 "world = first cam0 frame". |
| ImuPropagator  | src/ekf/imu_propagator.cpp      | IMU 측정값으로 nominal state 와 공분산 P 전파 (RK4 + 1차 Euler).                            |
| LcEKF          | src/ekf/lc_ekf.cpp              | ImuPropagator 를 wrap 하고 VO 추정으로 보정. Joseph form 공분산 업데이트.                       |
| run_vio.cpp    | apps/run_vio.cpp                | 메인 루프: yaml 로드 → reader 생성 → 초기화 → IMU/Cam interleave → 결과 저장.                  |

### 3.4 데이터 구조

include/core/types.hpp:1-37:

```

1 struct ImuData {
2     double timestamp;           // seconds
3     Eigen::Vector3d gyro;       // rad/s, body (IMU) frame
4     Eigen::Vector3d accel;      // m/s^2, body (IMU) frame
5 };
6
7 struct CamData {
8     double timestamp;
9     std::string img_l;          // 좌측 png 절대경로
10    std::string img_r;          // 우측 png 절대경로
11 };
12
13 struct GtData {
14     double timestamp;
15     Eigen::Vector3d p;          // world frame 위치
16     Eigen::Quaterniond q;       // world ← body 쿼터니언
17     Eigen::Vector3d v;          // world frame 속도
18 };
19
20 struct CameraParams {
21     double fx, fy, cx, cy;      // intrinsics
22     double k1, k2, p1, p2;      // radtan distortion
23     Eigen::Matrix4d T_cam_imu;  // IMU → cam (4x4)
24     int width, height;
25 };
26
27 struct ImuNoiseParams {
28     double gyro_noise = 1.6968e-4; // sigma_g [rad/s/sqrt(Hz)]
29     double accel_noise = 2.0000e-3; // sigma_a
30     double gyro_walk = 1.9393e-5;   // sigma_bg
31     double accel_walk = 3.0000e-3;   // sigma_ba
32 };

```

Listing 3.1: 핵심 struct

### 3.5 좌표계 5가지

1. **World (W)**: 첫 IMU 샘플 시점에서의 IMU body frame. 원점 (0,0,0), 중력은  $-Z$ .
2. **Body / IMU (I)**: 차량 IMU 좌표계. forward-X / left-Y / up-Z.
3. **Cam0 / Cam1 (C)**: 카메라 좌표계. right-X / down-Y / forward-Z (OpenCV 표준).
4. **Velodyne (V)**: 본 프로젝트 안 씬.  $T_{\text{cam,velo}}$  만 calibration 추출 시 잠시 사용.
5. **VO local**: StereoTracker 가 첫 cam0 프레임을 원점 (0,0,0) 으로 내부적으로 들고 다님. World 와 정렬하려면  $R_{WC_0}, t_{WC_0}$  변환 필요.

**Lever arm**:  $p_{IC} = T_{IC}[0;1]$  의 처음 3개 성분. IMU 좌표계에서 본 cam0 원점 위치. EKF 측정 모델의 핵심.

## 제 4 장

# 모듈별 코드 분석

이 장은 3개 코어 모듈과 2개 운용 모듈을 줄 단위로 분석하고 앞 장의 수식과 1:1 매핑한다.

### 4.1 ImuPropagator — IMU 전파

#### 4.1.1 생성자: 초기 분산 설정

```
1 ImuPropagator::ImuPropagator(const Vector3d& p0, const Vector3d& v0,
2                               const Matrix3d& R0, const Vector3d& bg0,
3                               const Vector3d& ba0,
4                               const ImuNoiseParams& noise,
5                               const Vector3d& g_world)
6 : p(p0), v(v0), R(R0), bg(bg0), ba(ba0), noise_(noise), g_(g_world)
7 {
8     P.setZero();
9     P.block<3,3>(0,0) = 1e-4 * Matrix3d::Identity(); // position
10    P.block<3,3>(3,3) = 1e-2 * Matrix3d::Identity(); // velocity
11    P.block<3,3>(6,6) = 1e-4 * Matrix3d::Identity(); // attitude
12    P.block<3,3>(9,9) = 1e-6 * Matrix3d::Identity(); // gyro bias
13    P.block<3,3>(12,12) = 1e-4 * Matrix3d::Identity(); // accel bias
14 }
```

Listing 4.1: src/ekf/imu\_propagator.cpp:9-25

#### 왜 이 값들?

- $1e^{-4} \text{ m}^2$  (위치):  $\sigma = 1 \text{ cm}$ . 첫 샘플은 "0 에 가깝다"고 강하게 믿음.
- $1e^{-2} \text{ m}^2/\text{s}^2$  (속도):  $\sigma = 0.1 \text{ m/s}$ . 차량이 정지일 가능성 "상당히" 믿음.
- $1e^{-4}$  (자세,  $\text{rad}^2$ ):  $\sigma \approx 0.01 \text{ rad} \approx 0.6^\circ$ . 정지 시 가속도계로 자세를 잡았으니 신뢰 높음.
- $1e^{-6}$  (gyro bias):  $\sigma = 0.001 \text{ rad/s}$ . 1초 평균 후 잔차 추정.
- $1e^{-4}$  (accel bias):  $\sigma = 0.01 \text{ m/s}^2$ . 비교적 큰 불확실 (가속도계 bias 는 잡기 어려움).

**튜닝 가능:** 시퀀스가 시작부터 움직이면 위치/속도/자세 분산을  $10\times$  키우는 게 발산 방지에 유리.

### 4.1.2 공개 API: propagate

```
1 void ImuPropagator::propagate(double t_new,
2                               const Vector3d& gyro_raw,
3                               const Vector3d& accel_raw) {
4     if (t_prev_ < 0.0) {
5         t_prev_ = t_new;
6         return; // 첫샘플은 dt 모름+스킵
7     }
8     double dt = t_new - t_prev_;
9     if (dt <= 0.0 || dt > 0.5) { // sanity gate
10         t_prev_ = t_new;
11         return;
12     }
13     t_prev_ = t_new;
14
15     Vector3d gyro_c = gyro_raw - bg; // bias 보정
16     Vector3d accel_c = accel_raw - ba;
17
18     rk4_step(dt, gyro_c, accel_c); // nominal state 적분
19     propagate_cov(dt, gyro_c, accel_c); // 공분산전파
20 }
```

Listing 4.2: imu\_propagator.cpp:129-149

왜 sanity gate  $dt > 0.5$  차단? 시퀀스 시작/종료 경계에서 timestamp 가 튀거나, 데이터 누락이 있을 때 큰  $dt$  로 적분하면  $O(dt^5)$  라도 거대한 위치 점프가 생긴다. 0.5 s 는 KITTI 의 정상 IMU 간격 0.1 s 의 5배라 안전 마진.

### 4.1.3 공분산 전파의 코드-수식 매핑

```
1 void ImuPropagator::propagate_cov(double dt,
2                                   const Vector3d& gyro_c,
3                                   const Vector3d& accel_c) {
4     Mat15 F = Mat15::Zero();
5     F.block<3,3>(0,3) = Matrix3d::Identity(); // (1)
6     F.block<3,3>(3,6) = -R * so3::hat(accel_c); // (2)
7     F.block<3,3>(3,12) = -R; // (3)
8     F.block<3,3>(6,6) = -so3::hat(gyro_c); // (4)
9     F.block<3,3>(6,9) = -R; // (5)
10
11     Mat15 Phi = Mat15::Identity() + F * dt; // (6)
12
13     Eigen::Matrix<double, 15, 12> G = ...::Zero();
14     G.block<3,3>(3,3) = R; // accel noise -> dv
15     G.block<3,3>(6,0) = R; // gyro noise -> dtheta
16     G.block<3,3>(9,6) = Matrix3d::Identity(); // gyro walk -> dbg
17     G.block<3,3>(12,9) = Matrix3d::Identity(); // accel walk -> dba
18
19     double sg=noise_.gyro_noise, sa=noise_.accel_noise;
20     double sbg=noise_.gyro_walk, sba=noise_.accel_walk;
21     Eigen::Matrix<double, 12, 12> Qc = ...::Zero();
22     Qc.block<3,3>(0,0) = sg*sg * Matrix3d::Identity();
23     Qc.block<3,3>(3,3) = sa*sa * Matrix3d::Identity();
24     Qc.block<3,3>(6,6) = sbg*sbg * Matrix3d::Identity();
```

```

25     Qc.block<3,3>(9,9) = sba*sba* Matrix3d::Identity();
26
27     Mat15 Qd = (G * Qc * G.transpose()) * dt;           // (7)
28     P = Phi * P * Phi.transpose() + Qd;                // (8)
29 }

```

Listing 4.3: imu\_propagator.cpp:89-125

매핑:

- (1)~(5): 공분산 전파 절의  $F$  행렬 5개 비영 블록과 정확히 일치.
- (6):  $\Phi = I + Fdt$  의 1차 Euler 이산화.
- (7):  $Q_d = GQ_cG^\top dt$ .  $\sigma$  들이 "밀도" 단위라  $dt$  곱.
- (8):  $P \leftarrow \Phi P \Phi^\top + Q_d$  의 EKF 표준식.

## 4.2 LcEKF — Loosely-Coupled 측정 업데이트

### 4.2.1 생성자와 lever arm

```

1 LcEKF::LcEKF(ImuPropagator& imu,
2             const Eigen::Matrix4d& T_cam0_imu,
3             double sigma_vo)
4 : imu_(imu), sigma_vo_(sigma_vo)
5 {
6     Eigen::Matrix4d T_imu_cam0 = T_cam0_imu.inverse();
7     p_IC_ = T_imu_cam0.block<3,1>(0,3); // body frame 에서본 cam0 원점
8 }

```

Listing 4.4: src/ekf/lc\_ekf.cpp:10-18

$T_{cam0,imu}$  는 IMU  $\rightarrow$  cam0 변환이므로 역행렬이 cam0  $\rightarrow$  IMU. 그 평행이동 부분이  $p_{IC}$  — body frame 에서 cam0 가 어디 있는지 (lever arm).

### 4.2.2 측정 업데이트 전체 코드

```

1 bool LcEKF::update_vo(const Vector3d& p_world_cam0) {
2     // (1) 예측측정값
3     const Matrix3d& R_wI = imu_.R;
4     const Vector3d& p_wI = imu_.p;
5     Vector3d h = R_wI * p_IC_ + p_wI;
6
7     // (2) Innovation
8     Vector3d innov = p_world_cam0 - h;
9
10    // (3) Measurement Jacobian (3x15)
11    Eigen::Matrix<double, 3, 15> H = ...::Zero();
12    H.block<3,3>(0,0) = Matrix3d::Identity();
13    H.block<3,3>(0,6) = so3::hat(R_wI * p_IC_);
14
15    // (4) Innovation covariance
16    Eigen::Matrix3d R_noise = (sigma_vo_*sigma_vo_) * Matrix3d::Identity();

```

```

17 Eigen::Matrix3d S = H * imu_.P * H.transpose() + R_noise;
18
19 // (5) Kalman gain
20 Eigen::Matrix<double, 15, 3> K = imu_.P * H.transpose() * S.inverse();
21
22 // (6) Error correction
23 Vec15 dx = K * innov;
24 apply_correction(dx);
25
26 // (7) Joseph-form covariance update
27 Mat15 I_KH = Mat15::Identity() - K * H;
28 imu_.P = I_KH * imu_.P * I_KH.transpose()
29         + K * R_noise * K.transpose();
30
31 if (innov.norm() > 5.0)
32     std::cerr << "[EKF] Large VO innovation: " << innov.norm() << " m\n";
33 return true;
34 }

```

Listing 4.5: lc\_ekf.cpp:26-68

### 4.2.3 상태 보정 적용

```

1 void LcEkf::apply_correction(const Vec15& dx) {
2     imu_.p += dx.segment<3>(0);
3     imu_.v += dx.segment<3>(3);
4     imu_.R = imu_.R * so3::Exp(dx.segment<3>(6));
5     imu_.R = Eigen::Quaterniond(imu_.R).normalized().toRotationMatrix();
6     imu_.bg += dx.segment<3>(9);
7     imu_.ba += dx.segment<3>(12);
8 }

```

Listing 4.6: lc\_ekf.cpp:70-77

**핵심:** 위치/속도/바이어스는  $+=$  (가법), 회전만  $R \cdot \text{Exp}(\delta\theta)$  합성. 그 후 정규화로 수치 오차 cosmetics.

## 4.3 StereoTracker — Stereo VO 프론트엔드

### 4.3.1 생성자: 캘리브레이션과 rectify 맵

```

1 StereoTracker::StereoTracker(const CameraParams& cam0,
2                               const CameraParams& cam1) {
3     // (1) Intrinsic 행렬과 distortion 벡터구성
4     K0_ = (cv::Mat_<double>(3,3) << cam0.fx, 0, cam0.cx, ...);
5     K1_ = ...
6
7     // (2) cam0 -> cam1 상대변환
8     T_cam1_cam0_ = T_C1_I * T_C0_I.inverse();
9     cv::Mat R10(3,3,CV_64F), t10(3,1,CV_64F); // T_cam1_cam0_ -> R10, t10
10
11     // (3) Stereo rectify (epipolar line 을수평으로 )
12     cv::stereoRectify(K0_, dist0_, K1_, dist1_, img_size,

```

```

13         R10, t10,
14         R0_rect_, R1_rect_, P0_, P1_, Q,
15         cv::CALIB_ZERO_DISPARITY, -1);
16
17 // (4) 맵셀별 undistort+rectify map 런타임에 ( cv::remap 으로적용 )
18 cv::initUndistortRectifyMap(K0_, dist0_, R0_rect_, P0_, img_size,
19                             CV_32FC1, map0x_, map0y_);
20 cv::initUndistortRectifyMap(K1_, dist1_, R1_rect_, P1_, img_size,
21                             CV_32FC1, map1x_, map1y_);
22
23 fx_rect_      = P0_.at<double>(0, 0);
24 baseline_rect_ = -P1_.at<double>(0, 3) / fx_rect_;
25 K_rect_      = P0_(cv::Rect(0, 0, 3, 3)).clone();
26
27 disparity_offset_px_ = -static_cast<float>(fx_rect_ * baseline_rect_ / 3.0);
28 orb_            = cv::ORB::create(1500, 1.2f, 8, 15, 0, 2,
29                                    cv::ORB::HARRIS_SCORE, 31, 7);
30 matcher_ = cv::BFMatcher::create(cv::NORM_HAMMING, false);
31 }

```

Listing 4.7: src/frontend/stereo\_tracker.cpp:16-91, 핵심부 발췌

핵심 결과:

- $P0_, P1_ :$  rectified projection matrix. KITTI 의 *calib* 와 일치해야 함.
- $\text{baseline\_rect}_\approx 0.537 \text{ m}$  — 미터 스케일의 출처.
- $\text{disparity\_offset\_px}_ = -f_x \cdot b/3$  — guided KLT 의 “예상 깊이 3m”에 해당하는 시차 초기 추정. 음수인 이유는 *cam1* 이 우측이라 *features* 가 왼쪽으로 이동.

### 4.3.2 한 프레임 처리

$N$ 번째 프레임에서 `tracker.process(img_l, img_r)` 호출 시:

1. **전처리:** BGR  $\rightarrow$  gray, *cv::remap* 으로 stereo rectify 적용.
2. **Bootstrap (첫 프레임만):** ORB 로 좌 $\leftrightarrow$ 우 매칭, 삼각측량, 깊이  $0.1 < Z < 50 \text{ m}$  만 keep, world frame = first cam0.
3. **시간축 추적:** KLT 로  $\text{prev\_left} \rightarrow \text{curr\_left}$ .
4. **스테레오 추적:** guided KLT 로  $\text{curr\_left} \rightarrow \text{curr\_right}$  (*disparity\_offset* 으로 초기 추정).
5. **Outlier 필터:** epipolar 제약 ( $|\Delta v| < 2 \text{ px}$ ), disparity 부호 검사.
6. **삼각측량:** 새 left-right 매칭으로 3D 점 갱신.
7. **PnP RANSAC:** 보유 3D 점과 *curr\_left* 픽셀로 카메라 6-DoF pose.
8. **Gap fill:** 추적 점이 *target\_features*<sub>=250</sub> 보다 적으면 FAST 검출 + ORB stereo 매칭으로 추가.
9. **상태 갱신:**  $\text{prev\_pts} \leftarrow \text{curr\_pts}$ .

## 4.4 KittiRawReader — 데이터 적재

### 4.4.1 디렉터리 구조 검증과 grayscale 우선

```
1 std::pair<fs::path, fs::path> select_stereo_roots(const fs::path& root) {
2     const std::pair gray{root / "image_00", root / "image_01"};
3     if (fs::exists(gray.first / "data") && fs::exists(gray.second / "data"))
4         return gray;
5
6     const std::pair color{root / "image_02", root / "image_03"};
7     if (fs::exists(color.first / "data") && fs::exists(color.second / "data"))
8         return color;
9
10    throw std::runtime_error("Cannot find KITTI stereo image folders ...");
11 }
```

Listing 4.8: src/io/kitti\_raw\_reader.cpp:79-91

**왜 grayscale 우선?** 컬러 카메라(image\_02/03)는 베이어 보간을 거쳐서 콘트라스트가 흑백 대비 약간 떨어진다. KLT 추적은 강한 그라디언트에 의존하므로 흑백이 안정적. 또한 KITTI의 흑백 카메라가 더 넓은 다이내믹 레인지.

### 4.4.2 OXTS 파싱: IMU + GT 한 번에

```
1 for (size_t i = 0; i < count; ++i) {
2     const auto values = load_numeric_tokens(files[i]);
3     if (values.size() < 23) continue;
4
5     const double lat = values[0];
6     const double lon = values[1];
7     const double alt = values[2];
8     const double roll = values[3];
9     const double pitch = values[4];
10    const double yaw = values[5];
11
12    if (!have_origin) {
13        mercator_scale = std::cos(lat * kPi / 180.0);
14        origin = Eigen::Vector3d(
15            mercator_x(lon, mercator_scale),
16            mercator_y(lat, mercator_scale),
17            alt);
18        have_origin = true;
19    }
20
21    ImuData imu;
22    imu.timestamp = timestamps[i];
23    imu.gyro = Eigen::Vector3d(values[20], values[21], values[22]); // wf, wl, ww
24    imu.accel = Eigen::Vector3d(values[14], values[15], values[16]); // af, al, au
25    imu_.push_back(imu);
26
27    GtData gt;
28    gt.timestamp = timestamps[i];
29    gt.p = Eigen::Vector3d(
30        mercator_x(lon, mercator_scale) - origin.x(),
```

```

31     mercator_y(lat, mercator_scale) - origin.y(),
32     alt - origin.z());
33 gt.q = Eigen::Quaterniond(rot_from_rpy(roll, pitch, yaw));
34 gt.q.normalize();
35 gt.v = Eigen::Vector3d(values[7], values[6], values[10]);
36 gt_.push_back(gt);
37 }

```

Listing 4.9: kitti\_raw\_reader.cpp:170-205, 핵심부

핵심:

- OXTS .txt 한 줄 30개 값에서 values[14..16] = (af, al, au) 가속도, values[20..22] = (wf, wl, wu) 각속도 *body frame, forward-left-up*.
- lat/lon/alt 는 첫 패킷을 원점으로 Mercator 투영해  $\mathbb{R}^3$  좌표화. (위경도 차이를 미터로 변환)
- GT 자세는 RPY  $\rightarrow$  쿼터니언.
- GT 속도는 north/east 가 values[6..7] 이고 east-north-up 순서로 저장 (코드의 values[7], [6], [10] 순서 확인).

## 4.5 run\_vio.cpp — 메인 루프

### 4.5.1 초기화: 정지 IMU 평균

```

1  InitialState initialize_from_imu(const std::vector<ImuData>& imu_data,
2                                double t0,
3                                const InitParams& init_params) {
4      InitialState init;
5      init.t0 = t0;
6      init.g_world = Eigen::Vector3d(0.0, 0.0, -init_params.gravity_norm);
7
8      Eigen::Vector3d gyro_sum = Eigen::Vector3d::Zero();
9      Eigen::Vector3d accel_sum = Eigen::Vector3d::Zero();
10     int sample_count = 0;
11
12     for (const auto& imu : imu_data) {
13         if (imu.timestamp < t0) continue;
14         if (imu.timestamp > t0 + init_params.static_window_sec) break;
15         gyro_sum += imu.gyro;
16         accel_sum += imu.accel;
17         ++sample_count;
18     }
19     // ...
20
21     init.bg0 = gyro_sum / sample_count;
22     const Vector3d accel_mean = accel_sum / sample_count;
23     init.R0 = estimate_initial_orientation(accel_mean);
24     return init;
25 }
26
27 Eigen::Matrix3d estimate_initial_orientation(const Vector3d& accel_mean) {
28     Eigen::Quaterniond q = Eigen::Quaterniond::FromTwoVectors(

```

```

29     accel_mean.normalized(),
30     Eigen::Vector3d::UnitZ()); // accel_mean -> +Z
31     return q.normalized().toRotationMatrix();
32 }

```

Listing 4.10: apps/run\_vio.cpp:96-155, 발췌

**왜 가속도 평균이 +Z 방향?** 정지 시 IMU 가속도계는  $R^T(-g) = R^T \cdot (0, 0, +9.81)^T$  를 출력 (중력의 반대 방향).  $R_0$  는 이 측정 벡터가 world +Z 가 되도록 회전을 잡는다.

**왜 1초 평균?** IMU 잡음은 white 라 평균이  $\sigma/\sqrt{N}$  로 줄어든다. KITTI 10 Hz 라  $N = 10$ ,  $\sqrt{10} \approx 3.2$ 배 잡음 감소.

#### 4.5.2 메인 처리 루프

```

1  for (const auto& cam : cam_data) {
2      if (max_frames > 0 && frame_count >= max_frames) break;
3
4      // (1) IMU 를 cam timestamp 까지모두전파
5      while (imu_idx < imu_data.size() &&
6             imu_data[imu_idx].timestamp <= cam.timestamp) {
7          const auto& imu = imu_data[imu_idx++];
8          ekf.propagate(imu.timestamp, imu.gyro, imu.accel);
9      }
10
11     // (2) 좌우/ png 로드
12     const cv::Mat img_l = cv::imread(cam.img_l, cv::IMREAD_COLOR);
13     const cv::Mat img_r = cv::imread(cam.img_r, cv::IMREAD_COLOR);
14     if (img_l.empty() || img_r.empty()) continue;
15
16     // (3) Stereo VO
17     const auto pose = tracker.process(img_l, img_r);
18
19     // (4) VO local -> world 변환
20     const Eigen::Vector3d p_world_cam = R_WC0 * pose.t + t_WC0;
21
22     // (5) EKF 측정보정첫 ( 프레임제외 )
23     if (pose.valid && frame_count > 0) {
24         ekf.update_vo(p_world_cam);
25     }
26
27     // (6) 결과누적
28     traj_est.push_back(ekf.position());
29     traj_ts.push_back(cam.timestamp);
30
31     Eigen::Vector3d gt_p;
32     if (sample_gt_position(gt_data, cam.timestamp, gt_p))
33         traj_gt.push_back(gt_p);
34
35     ++frame_count;
36 }

```

Listing 4.11: run\_vio.cpp:310-340

### 4.5.3 ATE 계산

```
1 double compute_ate(const std::vector<Vector3d>& est,
2                   const std::vector<Vector3d>& gt) {
3     if (est.size() != gt.size() || est.empty()) return -1.0;
4
5     Eigen::MatrixXd est_mat(3, est.size());
6     Eigen::MatrixXd gt_mat (3, gt.size());
7     for (size_t i = 0; i < est.size(); ++i) {
8         est_mat.col(i) = est[i];
9         gt_mat .col(i) = gt[i];
10    }
11
12    const Eigen::Matrix4d T_gt_est =
13        Eigen::umeyama(est_mat, gt_mat, /*with_scale=*/false);
14
15    double sum2 = 0.0;
16    for (size_t i = 0; i < est.size(); ++i) {
17        Eigen::Vector4d p_est(est[i].x(), est[i].y(), est[i].z(), 1.0);
18        Eigen::Vector3d aligned = (T_gt_est * p_est).head<3>();
19        sum2 += (aligned - gt[i]).squaredNorm();
20    }
21    return std::sqrt(sum2 / est.size());
22 }
```

Listing 4.12: run\_vio.cpp:182-200

with\_scale=false 로 스테레오의 미터 스케일을 보존하면서 회전+평행이동만 정합.

## 제 5 장

# 운용 절차 — 데이터부터 결과까지

### 5.1 사전 요구사항

| 항목     | 권장 사양                     | 비고             |
|--------|---------------------------|----------------|
| OS     | WSL2 Ubuntu 22.04 / Linux | macOS 미검증      |
| 컴파일러   | gcc 10+ (C++20)           | gcc --version  |
| CMake  | 3.16+                     |                |
| 디스크    | 단일 drive 0.5~5 GB         | 전체 raw ~180 GB |
| 메모리    | 8 GB+                     |                |
| Python | 3.8+ + numpy/matplotlib   | 시각화용           |

### 5.2 의존성 설치

```
1 sudo apt update
2 sudo apt install -y \
3     build-essential cmake git pkg-config \
4     libeigen3-dev libopencv-dev libyaml-cpp-dev \
5     python3 python3-pip python3-numpy python3-matplotlib
```

버전 점검:

```
1 gcc --version                # 10.x 이상
2 pkg-config --modversion eigen3 # 3.3+
3 pkg-config --modversion opencv4 # 4.x
4 pkg-config --modversion yaml-cpp # 0.6+
```

### 5.3 KITTI raw 다운로드

1. [https://www.cvlibs.net/datasets/kitti/raw\\_data.php](https://www.cvlibs.net/datasets/kitti/raw_data.php) 에서 계정 등록 (이메일 인증 필수).
2. 시퀀스마다 2개 zip 다운:
  - 2011\_09\_26\_calib.zip (~1 MB) — 날짜당 1회
  - 2011\_09\_26\_drive\_XXXX\_sync.zip — 메인 데이터
3. LiDAR/tracklets/extract 는 받지 말 것 (안 씀).

### 5.3.1 추천 시퀀스

| drive                 | 카테고리        | 길이      | 권장 용도                 |
|-----------------------|-------------|---------|-----------------------|
| 2011_09_26_drive_0001 | residential | 11 s    | 디버그                   |
| 2011_09_26_drive_0009 | residential | 47 s    | 기본 검증                 |
| 2011_09_26_drive_0117 | city        | 1 m 6 s | City 최장               |
| 2011_09_30_drive_0028 | residential | 8 m     | KITTI Odometry seq 08 |
| 2011_10_03_drive_0027 | residential | 7 m     | KITTI Odometry seq 00 |

### 5.3.2 디렉터리 구조

```

1 data/kitti_raw/ └──
2   2011_09_26/ └──
3     calib_cam_to_cam.txt └──
4     calib_imu_to_velo.txt └──
5     calib_velo_to_cam.txt └──
6     2011_09_26_drive_0001_sync/ └──
7       image_00/{data/, timestamps.txt} └──
8       image_01/, image_02/, image_03/ └──
9       oxts/{data/, timestamps.txt, dataformat.txt}

```

## 5.4 캘리브레이션 추출

KITTI의 calib 3종에서 본 프로젝트가 요구하는  $T_{\text{cam},\text{imu}}$   $4 \times 4$  두 개 (cam0, cam1)를 계산.

### 5.4.1 Chain rule

$$T_{\text{cam0},\text{imu}} = R_{\text{rect},00} \cdot T_{\text{cam},\text{velo}} \cdot T_{\text{velo},\text{imu}} \quad (5.1)$$

cam1은 추가로:

$$T_{\text{cam1},\text{imu}} = T_{\text{cam1},\text{cam0}} \cdot T_{\text{cam0},\text{imu}} \quad (5.2)$$

$T_{\text{cam1},\text{cam0}}$ 의 평행이동  $t_x = -\text{baseline}$  (rectified 좌표계에서 cam1은 cam0의  $-X$  방향).

### 5.4.2 Python 스크립트

```

1 from pathlib import Path
2 import numpy as np
3
4 CALIB = Path("/mnt/d/02_research/06_cpp_LC-EKF_VIO_kitti_raw/data/kitti_raw/2011_09_26")
5
6 def parse_kv(path):
7     out = {}
8     for line in path.read_text().splitlines():
9         if ":" not in line: continue
10        k, v = line.split(":", 1)
11        out[k.strip()] = np.fromstring(v, sep=" ")
12    return out
13
14 def homog(R, t):

```

```

15     T = np.eye(4); T[:3,:3] = R.reshape(3,3); T[:3,3] = t.reshape(3); return T
16
17     cam = parse_kv(CALIB / "calib_cam_to_cam.txt")
18     v2c = parse_kv(CALIB / "calib_velo_to_cam.txt")
19     i2v = parse_kv(CALIB / "calib_imu_to_velo.txt")
20
21     T_cam_velo = homog(v2c["R"], v2c["T"])
22     T_velo_imu = homog(i2v["R"], i2v["T"])
23     R_rect_00 = np.eye(4); R_rect_00[:3,:3] = cam["R_rect_00"].reshape(3,3)
24
25     T_cam0_imu = R_rect_00 @ T_cam_velo @ T_velo_imu
26
27     P_rect_00 = cam["P_rect_00"].reshape(3,4)
28     P_rect_01 = cam["P_rect_01"].reshape(3,4)
29     fx = P_rect_00[0,0]; fy = P_rect_00[1,1]
30     cx = P_rect_00[0,2]; cy = P_rect_00[1,2]
31     baseline_x = -P_rect_01[0,3] / fx
32     T_cam1_cam0 = np.eye(4); T_cam1_cam0[0,3] = -baseline_x
33     T_cam1_imu = T_cam1_cam0 @ T_cam0_imu
34
35     w, h = cam["S_rect_00"].astype(int)
36     print(f"{int(w)}x{int(h)}, fx={fx:.4f}, fy={fy:.4f}, cx={cx:.4f}, cy={cy:.4f}")
37     print(f"baseline_x = {baseline_x:.6f}")

```

Listing 5.1: 임시 파일로 저장 후 1회 실행

## 5.5 YAML 설정

```

1  dataset_type: "kitti_raw"
2  dataset: "/mnt/d/.../data/kitti_raw/2011_09_26/2011_09_26_drive_0001_sync"
3  output:  "/mnt/d/.../results/kitti_raw_2011_09_26_drive_0001"
4  max_frames: 0          # 0 = 전체
5
6  init:
7      mode: "stationary_imu"
8      static_window_sec: 1.0
9      gravity_norm: 9.81
10
11  imu_noise:
12      gyro:      1.6968e-4
13      accel:      2.0000e-3
14      gyro_walk:  1.9393e-5
15      accel_walk: 3.0000e-3
16
17  ekf:
18      sigma_vo: 0.10      # VO 측정잡음 [m]
19
20  cam0:
21      width: 1242
22      height: 375
23      fx: 721.5377
24      fy: 721.5377
25      cx: 609.5593
26      cy: 172.8540

```

```

27 k1: 0.0 # KITTI rect 이미지는왜곡 0
28 k2: 0.0
29 p1: 0.0
30 p2: 0.0
31 T_cam_imu:
32   - [ 0.000998747206, -0.999990382071, 0.004259378488, -0.314076869806 ]
33   - [ 0.008416901828, -0.004250821136, -0.999955569675, 0.719452035633 ]
34   - [ 0.999964048697, 0.001034553284, 0.008412575209, -1.089082940192 ]
35   - [ 0.0, 0.0, 0.0, 1.0 ]
36
37 cam1:
38   # 같은 intrinsics, T_cam_imu 만 baseline 만큼다름
39   ...

```

Listing 5.2: config/kitti\_raw\_2011\_09\_26\_drive\_0001.yaml

## 5.6 빌드 & 실행

```

1 cd /mnt/d/02_research/06_cpp_LC-EKF_VIO_kitti_raw
2 mkdir -p build && cd build
3 cmake .. -DCMAKE_BUILD_TYPE=Release
4 make -j$(nproc)
5
6 cd ..
7 ./build/run_vio config/kitti_raw_2011_09_26_drive_0001.yaml

```

정상 stdout (drive\_0001 기준):

```

1 [KITTI raw] IMU=108 CAM=108 GT=108
2 Init window samples=11 mean accel= ... 9.8...
3 Init gravity= 0 0 -9.81
4 Frame 100 ... tracked=...
5 =====
6 Frames processed : 108
7 ATE RMSE : 156.3 cm
8 =====

```

## 5.7 시각화

```

1 python3 tools/plot_trajectory.py results/kitti_raw_2011_09_26_drive_0001/

```

생성된 trajectory\_plot.png: 왼쪽 XY top-down (estimated vs GT), 오른쪽 XYZ vs time.

## 제 6 장

# 다중 시퀀스 일괄 테스트

### 6.1 단일 날짜 (sed 복제)

```
1 BASE=config/kitti_raw_2011_09_26_drive_0001.yaml
2 for drv in 0009 0117 0009 0036 0086; do
3     NEW=config/kitti_raw_2011_09_26_drive_${drv}.yaml
4     sed \
5         -e "s|drive_0001_sync|drive_${drv}_sync|g" \
6         -e "s|kitti_raw_2011_09_26_drive_0001|kitti_raw_2011_09_26_drive_${drv}|g" \
7         "$BASE" > "$NEW"
8 done
```

### 6.2 전체 KITTI raw 자동화 (5개 날짜)

전체 데이터셋(5 calib 날짜)을 한 번에 다루려면 캘리브레이션 추출 절의 Python 스크립트를 확장해 모든 2011\_\*\_drive\_\*\_sync 디렉터리에 대해 yaml 자동 생성. (스크립트는 부록 A 참조)

### 6.3 일괄 실행

```
1 mkdir -p logs
2 for cfg in config/kitti_raw_2011_*_drive_*.yaml; do
3     name=$(basename "$cfg" .yaml)
4     ./build/run_vio "$cfg" 2>&1 | tee "logs/${name}.log"
5 done
```

### 6.4 결과 집계

```
1 echo "date,drive,frames,ate_rmse_m" > summary.csv
2 for m in results/kitti_raw_2011_*_drive_*/metrics.txt; do
3     name=$(basename $(dirname "$m"))
4     date=$(echo "$name" | sed 's/kitti_raw_//;s/_drive_./../')
5     drv=$(echo "$name" | sed 's/.*_drive_../')
6     f=$(awk -F': ' '/^frames/{print $2}' "$m")
7     a=$(awk -F': ' '/^ate_rmse_m/{print $2}' "$m")
8     echo "${date},${drv},${f},${a}" >> summary.csv
9 done
```

## 제 7 장

# 결과 해석 & 트러블슈팅

### 7.1 실패 모드 진단표

| 증상                            | 원인                       | 처방                               |
|-------------------------------|--------------------------|----------------------------------|
| tracked=0 빈번                  | 어두운 시퀀스 / 빠른 회전 / KLT 손실 | grayscale 강제, drive 변경           |
| EKF 발산 (frame 100 후 imu_p 폭주) | sigma_vo 너무 작음           | 0.10 → 0.20                      |
| Init 직후부터 어긋남                 | static_window 안에 차량 출발   | 0.3~0.5초로 단축                     |
| ATE unavailable               | GT 로드 실패                 | oxts/timestamps.txt 존재 확인        |
| GT 와 회전된 채 평행이동               | LC-EKF 글로벌 yaw 미관측       | 정상 한계, Tight MSCKF 필요            |
| 중간 점프                         | VO outlier               | sigma_vo 상향 + chi-square gate 추가 |
| Z축 폭주                         | 초기 gravity 추정 실패         | static_window 단축                 |
| 모든 프레임 같은 위치                  | EKF propagate 누락         | timestamp 단위 확인                  |

### 7.2 LC-EKF 본질적 한계

1. VO 누적 드리프트가 EKF 안으로 그대로 들어옴.
2. Yaw 절대 관측 부재 — 글로벌 yaw drift 보정 불가.
3. Feature-level 기하 제약 활용 불가.

이 한계를 넘으려면 Tight MSCKF (OpenVINS 류) 로 가야 한다.

## 제 8 장

# 수정 시나리오 5선

### 8.1 VO 신뢰도 ↑

```
1 # config/...drive_X.yaml
2 ekf:
3   sigma_vo: 0.05      # 0.10 -> 0.05 잡음 ( 1/2)
```

효과:  $R_{\text{noise}} 1/4 \rightarrow K$  증가  $\rightarrow$  VO 가 EKF 를 더 강하게 끌어당김. ATE 보통 줄지만 outlier 많으면 발산 위험.

### 8.2 초기 자세 불확실성 ↑

```
1 // src/ekf/imu_propagator.cpp:22
2 P.block<3,3>(6,6) = 1e-2 * Matrix3d::Identity(); // 1e-4 -> 1e-2
```

효과: 초기 attitude 분산 100배  $\rightarrow$  첫 EKF 업데이트에서 회전 보정량 큼. 정지 구간에서 흔들림 가능, 1e-3 권장.

### 8.3 특징점 풀 확장

```
1 // include/frontend/stereo_tracker.hpp:57
2 int target_features_ = 400; // 250 -> 400
```

효과: KLT 추적 풀 확장  $\rightarrow$  PnP 인라이어 두꺼워짐. CPU 시간 비례 증가.

### 8.4 Chi-square outlier gate

```
1 Vector3d innov = p_world_cam0 - h;
2
3 // chi-square 게이트
4 Eigen::Matrix<double, 3, 15> H_tmp = ...::Zero();
5 H_tmp.block<3,3>(0,0) = Matrix3d::Identity();
6 H_tmp.block<3,3>(0,6) = so3::hat(R_wI * p_IC_);
7 Eigen::Matrix3d S_tmp = H_tmp * imu_.P * H_tmp.transpose()
8   + (sigma_vo*sigma_vo_) * Matrix3d::Identity();
9 double chi2 = innov.transpose() * S_tmp.inverse() * innov;
```

```

10 const double GATE_3DOF_99 = 11.345; //  $\chi^2$  (df=3, p=0.99)
11 if (chi2 > GATE_3DOF_99) {
12     std::cerr << "[EKF] gated VO innov: chi2=" << chi2 << "\n";
13     return false;
14 }

```

Listing 8.1: src/ekf/lc\_ekf.cpp innovation 직후

## 8.5 GPS 측정 추가

```

1 bool LcEkf::update_gps(const Vector3d& p_gps, double sigma_gps) {
2     Vector3d h = imu_.p; //  $h(x) = p_w I$ 
3     Vector3d innov = p_gps - h;
4     Eigen::Matrix<double, 3, 15> H = ...::Zero();
5     H.block<3,3>(0,0) = Matrix3d::Identity(); // VO 와 다르게  $d\theta$  영향없음
6     Eigen::Matrix3d R_n = (sigma_gps*sigma_gps) * Matrix3d::Identity();
7     Eigen::Matrix3d S = H * imu_.P * H.transpose() + R_n;
8     Eigen::Matrix<double, 15, 3> K = imu_.P * H.transpose() * S.inverse();
9     Vec15 dx = K * innov;
10    apply_correction(dx);
11    Mat15 I_KH = Mat15::Identity() - K * H;
12    imu_.P = I_KH * imu_.P * I_KH.transpose() + K * R_n * K.transpose();
13    return true;
14 }

```

Listing 8.2: 새 메서드 LcEkf::update\_gps

## 제 9 장

# Knob 카탈로그

### 9.1 YAML knobs

| knob                       | 코드 위치                      | 영향        |
|----------------------------|----------------------------|-----------|
| ekf.sigma_vo               | run_vio.cpp:271            | VO 신뢰도    |
| imu_noise.gyro/accel       | run_vio.cpp:68-74, 269     | 잡음 가정     |
| imu_noise.*_walk           | 동상                         | bias 드리프트 |
| init.static_window_sec     | run_vio.cpp:80-83, 123-125 | 초기 평균화    |
| init.gravity_norm          | run_vio.cpp:80-83, 117     | 중력 크기     |
| max_frames                 | run_vio.cpp:259, 311       | 부분 실행     |
| cam0/cam1.fx..p2,T_cam_imu | run_vio.cpp:43-63          | 캘리브레이션    |

### 9.2 C++ 상수 knobs

| knob                 | 위치                         | 현재값        |
|----------------------|----------------------------|------------|
| P 초기 분산 (위치)         | imu_propagator.cpp:20      | 1e-4       |
| P 초기 분산 (속도)         | imu_propagator.cpp:21      | 1e-2       |
| P 초기 분산 (자세)         | imu_propagator.cpp:22      | 1e-4       |
| P 초기 분산 (gyro bias)  | imu_propagator.cpp:23      | 1e-6       |
| P 초기 분산 (accel bias) | imu_propagator.cpp:24      | 1e-4       |
| dt sanity gate       | imu_propagator.cpp:137     | 0.5 s      |
| target_features_     | stereo_tracker.hpp:57      | 250        |
| min_pnp_inliers_     | stereo_tracker.hpp:58      | 10         |
| disparity_offset_px_ | stereo_tracker.cpp:80      | $-f_x b/3$ |
| ORB nfeatures        | stereo_tracker.cpp:85      | 1500       |
| 삼각측량 깊이 게이트          | stereo_tracker.cpp:118-119 | 0.1~50 m   |
| 큰 innov 경고           | lc_ekf.cpp:64-65           | 5.0 m      |

## 부록 A

# 명령어 치트시트

```
1 # 의존성
2 sudo apt install -y build-essential cmake libeigen3-dev libopencv-dev libyaml-cpp-dev
   python3-numpy python3-matplotlib
3
4 # 빌드
5 cd /mnt/d/02_research/06_cpp_LC-EKF_VIO_kitti_raw
6 mkdir -p build && cd build && cmake .. -DCMAKE_BUILD_TYPE=Release && make -j$(nproc)
7
8 # 단일실행
9 ./run_vio ../config/kitti_raw_2011_09_26_drive_0001.yaml
10
11 # 시각화
12 cd .. && python3 tools/plot_trajectory.py results/kitti_raw_2011_09_26_drive_0001/
13
14 # 일괄실행
15 mkdir -p logs
16 for cfg in config/kitti_raw_2011*_drive*.yaml; do
17     name=$(basename "$cfg" .yaml)
18     ./build/run_vio "$cfg" 2>&1 | tee "logs/${name}.log"
19 done
20
21 # 결과요약
22 echo "date,drive,frames,ate_rmse_m" > summary.csv
23 for m in results/kitti_raw_2011*_drive*/metrics.txt; do
24     name=$(basename $(dirname "$m"))
25     d=$(echo "$name" | sed 's/kitti_raw_//;s/_drive_.*//')
26     drv=$(echo "$name" | sed 's/_drive_//')
27     f=$(awk -F': ' '/^frames/{print $2}' "$m")
28     a=$(awk -F': ' '/^ate_rmse_m/{print $2}' "$m")
29     echo "${d},${drv},${f},${a}" >> summary.csv
30 done
```

## 부록 B

### 행렬 도감

전체 시스템에 등장하는 모든 주요 행렬:

|                      | 이름              | 크기                       | 위치 | 의미                                  |
|----------------------|-----------------|--------------------------|----|-------------------------------------|
| $T_{\text{cam,imu}}$ | $4 \times 4$    | yaml                     |    | IMU $\rightarrow$ cam 변환            |
| $p_{IC}$             | 3               | lc_ekf.cpp:17            |    | body frame 의 cam0 위치                |
| $R_{WC_0}, t_{WC_0}$ | $3 \times 3, 3$ | run_vio.cpp:285-288      |    | VO local $\rightarrow$ world anchor |
| $R$                  | $3 \times 3$    | imu_propagator.cpp       |    | world $\leftarrow$ IMU body 자세      |
| $P$                  | $15 \times 15$  | :18-24, :124, lc_ekf:60  |    | error-state 공분산                     |
| $F$                  | $15 \times 15$  | imu_propagator.cpp:93-99 |    | 연속 전이 행렬                            |
| $G$                  | $15 \times 12$  | :106-110                 |    | 잡음 $\rightarrow$ 상태 매핑              |
| $Q_c$                | $12 \times 12$  | :115-119                 |    | 연속 잡음 밀도                            |
| $\Phi = I + Fdt$     | $15 \times 15$  | :102                     |    | 이산 전이                               |
| $Q_d$                | $15 \times 15$  | :122                     |    | 이산 잡음                               |
| $H$                  | $3 \times 15$   | lc_ekf.cpp:41-43         |    | 측정 야코비안                             |
| $R_{\text{noise}}$   | $3 \times 3$    | :46                      |    | VO 측정 잡음                            |
| $S$                  | $3 \times 3$    | :47                      |    | innovation 공분산                      |
| $K$                  | $15 \times 3$   | :50                      |    | Kalman gain                         |
| $\delta x$           | 15              | :53                      |    | error-state 보정                      |
| $T_{\text{gt,est}}$  | $4 \times 4$    | run_vio.cpp:193          |    | Umeyama 정합                          |
| $P_0, P_1$           | $3 \times 4$    | stereo_tracker.cpp:57-67 |    | rectified projection                |
| $K_{\text{rect}}$    | $3 \times 3$    | 동상                       |    | rectified intrinsics                |